

1. Illustrate the concept of players of a two-player board game. (3)

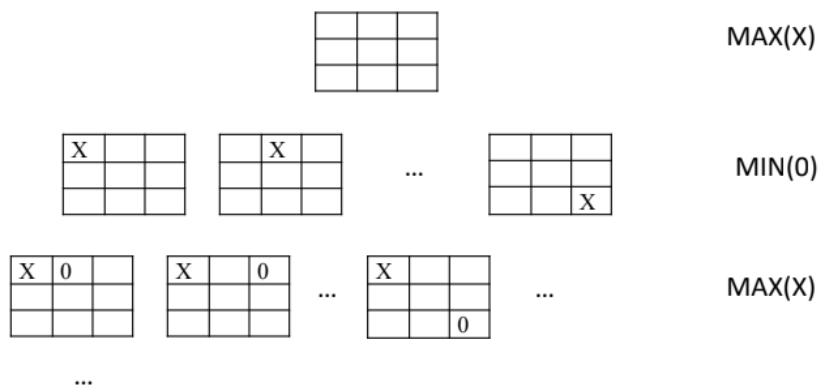
Ans :

A game consists of a set of two or more players, a set of moves for the players, and a specification of payoffs (outcomes) for each combination of strategies. A strategy defines a complete plan of action for a given player.

A) Representation of a Two-player Board Game

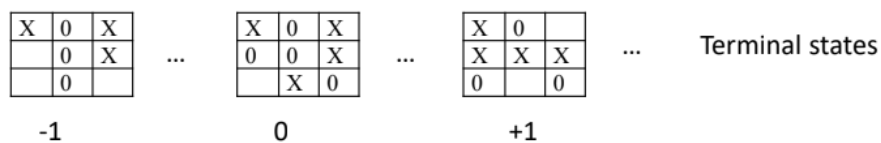
i) Players: MAX & MIN; Anyone can start; Make moves alternating each other.

ii) Classroom example: Tic-Tac-Toe



-Aug-21

1



[Values returned by Utility function / Payoff function/
Objective function; MAX's point of view has been reflected.]

iv) Strategies of Players

- ✓ MAX searches for the sequence of moves (Plan) that leads to a terminal with maximum possible utility value, even if MIN plays in the best way;
- ✓ MIN searches for the opposite, that is, terminal with minimum possible utility.

2. Why are game playing problems considered adversarial search problems? (2)

Ans :

Adversarial search is search when there is an "enemy" or "opponent" changing the state of the problem every step in a direction you do not want. The environment with more than one agent is termed a multi-agent environment, in which each agent is an opponent of another agent and playing against each other. Each agent needs to consider the action of another agent and the effect of that action on their performance. So, Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called adversarial searches, often known as Games.

3. What is a partial-order plan? (1)

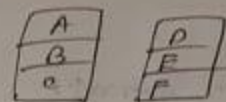
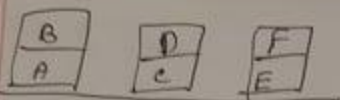
Ans :

Partial-order planning is an approach to automated planning that maintains a partial ordering between actions and only commits ordering between actions when forced to, that is, ordering of actions is partial.

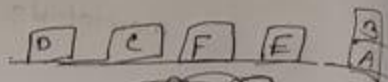
4. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in time over gain in actions. (4)

Ans :





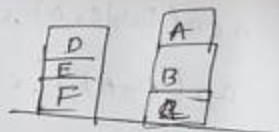
Action 1: move to Table (D, C)
move to Table (F, E)



Action 2: move (B, A, C)
move (E, table, F)



Action 3: move (D, table, E)
move (A, table, B)



Initial state: $\neg \text{on}(\text{table}, A) \wedge \neg \text{on}(\text{table}, C) \wedge \neg \text{on}(\text{table}, E)$
 $\wedge \text{on}(A, B) \wedge \text{on}(C, D) \wedge \text{on}(E, F) \wedge \text{clear}(B) \wedge \text{clear}(D)$
 $\wedge \text{clear}(F) \wedge \text{on}(\text{table}, D) \wedge \text{on}(\text{table}, F)$
 $\wedge \text{clear}(C) \wedge \text{clear}(E) \wedge \text{on}(C, B) \wedge \text{on}(E, F)$
 $\wedge \text{on}(E, D) \wedge \text{on}(B, A)$

1. Explain the concept of action schemas using an example from the Air Cargo Transportation problem. (3)

Ans:

Action Schemas:

A set of ground (variable-free) actions can be represented by a single action schema.

The schema is a lifted representation—it lifts the level of reasoning from propositional logic

to a restricted subset of first-order logic. The schema consists of the action name, a list of all the variables used in the schema, precondition and an effect.

i) Action: Load(c, p, a)

Precondition: $\text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a) \wedge \text{At}(a, c) \wedge \text{At}(a, p)$

Effect: $\text{At}(a, c) \wedge \neg \text{In}(p, c)$

ii) Action: Unload(c, p, a)

Precondition: $\text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a) \wedge \text{At}(a, p) \wedge \neg \text{In}(p, c)$

Effect: $\neg \text{At}(a, c) \wedge \text{In}(p, c)$

iii) Action: Fly(p, from, to)

Precondition: $\text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to}) \wedge \text{At}(\text{from}, p)$

Effect: $\neg \text{At}(\text{from}, p) \wedge \text{At}(\text{to}, p)$

$\text{Init}(\text{At}(C1, \text{SFO}) \wedge \text{At}(C2, \text{JFK}) \wedge \text{At}(P1, \text{SFO}) \wedge \text{At}(P2, \text{JFK}) \wedge \text{Cargo}(C1) \wedge \text{Cargo}(C2) \wedge \text{Plane}(P1) \wedge \text{Plane}(P2) \wedge \text{Airport}(\text{JFK}) \wedge \text{Airport}(\text{SFO}))$

$\text{Goal}(\text{At}(C1, \text{JFK}) \wedge \text{At}(C2, \text{SFO}))$ Action(Load(c, p, a),

PRECOND: $\text{At}(c, a) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\neg \text{At}(c, a) \wedge \text{In}(c, p)$) Action(Unload(c, p, a),

PRECOND: $\text{In}(c, p) \wedge \text{At}(p, a) \wedge \text{Cargo}(c) \wedge \text{Plane}(p) \wedge \text{Airport}(a)$

EFFECT: $\text{At}(c, a) \wedge \neg \text{In}(c, p)$) Action(Fly(p, from, to),

PRECOND: $\text{At}(p, \text{from}) \wedge \text{Plane}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$

EFFECT: $\neg \text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$)

The goal is just like a precondition: a conjunction of literals (positive or negative) that may contain variables, such as $\text{At}(p, \text{SFO}) \wedge \text{Plane}(p)$. Any variables are treated as existentially quantified, so this goal is to have any plane at SFO. The problem is solved when we can find a sequence of actions that end in a state s that entails the goal.

2. How a planner exerts intelligence through decomposition? (2)

Ans :

The problem solver might be inefficient because it cannot take advantage of problem decomposition. Consider the problem of delivering a set of overnight packages to their respective destinations, which are scattered across Australia. It makes sense to find out the nearest airport for each destination and divide the overall problem into several subproblems, one for each airport. Within the set of packages routed through a given airport, whether further decomposition is possible depends on the destination city. We saw in Chapter 5 that the ability to do this kind of decomposition contributes to the efficiency of constraint satisfaction problem solvers. The same holds true for planners: in the worst case, it can take $O(n!)$ time to find the best plan to deliver n packages, but only $O((n/k)! \times k)$ time if the problem can be decomposed into k equal parts

3. What is singular expansion? (1)

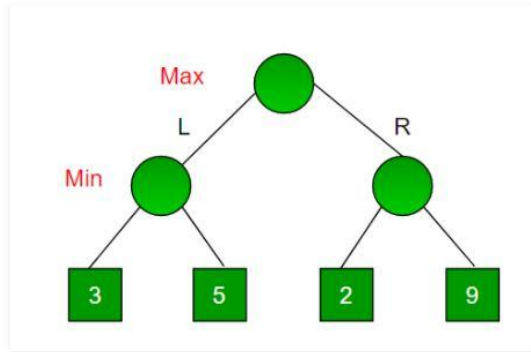
Ans : Square integrable kernel K holds

$$K(s, t) = \sum_{i=1}^{\infty} \mu_i u_i(s) v_i(t),$$

where $\langle u_i, u_j \rangle = \langle v_i, v_j \rangle = \delta_{ij}$, and $\mu_1 \geq \mu_2 \geq \mu_3 \geq \dots \geq 0$.

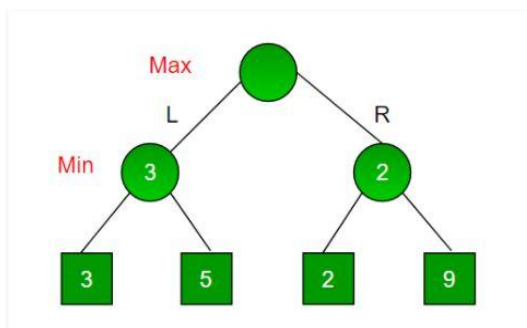
4. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [5, 3, 2, 4, 1, 3, 6, 2, 8, 7, 5, 1, 3, 4] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MIN makes the first move, and nodes are generated from left to right. (4)

Ans :



Since this is a backtracking based algorithm, it tries all possible moves, then backtracks and makes a decision.

- Maximizer goes LEFT: It is now the minimizers turn. The minimizer now has a choice between 3 and 5. Being the minimizer it will definitely choose the least among both, that is 3
- Maximizer goes RIGHT: It is now the minimizers turn. The minimizer now has a choice between 2 and 9. He will choose 2 as it is the least among the two values.
- Being the maximizer you would choose the larger value that is 3. Hence the optimal move for the maximizer is to go LEFT and the optimal value is 3. Now the game tree looks like below :



2. How imperfect information plays a role in game playing? (2)

Ans :

"What a player knows" is reflected in the strategies available to the player, which are determined by the information sets. If two nodes x and y are in the same information set, then the player "cannot tell them apart," because by definition the player's strategy must be the same (choose the j th child, for some fixed j) on the two nodes. Thus, when there exists an information set with more than one node in it, the game is said to exhibit imperfect information.

Solving games with imperfect information is NPhard (in contrast to games with perfect information), even when there is only a single player

We give an algorithm called IMP-minimax (for "imperfect information minimax") that is the natural analog to chance-minimax, but designed to reflect the presence of information sets. IMP-minimax computes a strategy for games with imperfect information in time linear in the size of the search tree. The strategy produced by this algorithm is guaranteed to be an optimal strategy, if the imperfect information game has a single player and a certain natural property called perfect recall.

3. What is an Add list? (1)

Ans : It is a list where the positive literals of an effect in the representation of goals using PDDL are being stored.

4. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates 'Airport', 'Cargo' and 'Plane', and consider generally accepted rules and notations for such a problem. (4)

Ans :



109

1. Explain the ways of improving the performance of the MINIMAX algorithm. (3)

Ans:

Common Factors:

- Database of experience like 'best opening move', 'great winning [/losing] combination', ...
- Maintaining transposition table that marks repeated states
- Forward pruning, that excludes equivalent states
- Singular expansion, where only 'clearly better' moves are considered

Using 'cutoff test' and heuristic evaluation function:

- To have a reduced search space 'cutoff tests' are applied in place of 'terminal tests':
 - 1.Simple: Set a fixed depth; Not so good.
 - 2.Sophisticated: quiescence search; [A quiescent state does not exhibit wild swing in near future.]
- To apply on states at cutoff depths, utility functions are replaced with heuristic functions, which return expected utility of states. [In Chess, material values like pawn = 1, knight/bishop = 3, queen = 9, etc. are used.]

Using alpha-beta pruning:

- Cutting off in compliance with already calculated minimax values, that is, values of
 - best choice for the maximizing player, alpha(eikhane alpha symbol dish)
 - best choice for the minimizing player, . beta(eikhane beta symbol dish)

2. How game playing exhibits features of search problems? (2)

Ans:

3. What are the components of an action schema? (1)

Ans: An action schema contains:

- An action name with a parameter list
- A 'precondition' as a conjunction of function-free and positive literals, where the arguments of the predicates are taken from the parameter list
- An 'effect' as a conjunction of function-free literals, where also the arguments of the predicates are taken from the parameter list

4. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in actions over gain in time. (4)



Action 1: moveToTable(A, D)
moveToTable(E, B)

A E B ^C F D

Action 2:
move(E, Table, D)
move(B, Table, A)

B E C
A D F

Action 3: move(C, F, B)

^C B E C
A D F

Action 4: move(F, table, E)

States: $\text{On}(\text{table}, B) \wedge \text{on}(\text{table}, F) \wedge \text{on}(\text{table}, D)$
 $\wedge \text{on}(B, E) \wedge \text{on}(F, C) \wedge \text{on}(D, A) \wedge \text{clear}(E) \wedge \text{clear}(C)$
 $\wedge \text{clear}(A) \wedge \text{non}(\text{table}, A) \wedge \text{on}(\text{table}, E) \wedge \text{clear}(B)$
 $\wedge \text{clear}(D)$
 $\wedge \text{on}(A, B) \wedge \text{on}(D, E) \wedge \text{on}(C, F)$
 $\wedge \text{on}(E, F)$

1. Explain how actions are carried out to get to the next state during planning. (3)

Ans :

- An action can be carried out on a state if the precondition holds on that state with appropriate substitutions
- If an action is carried out, the Add-list elements are included in the state, while the Delete- list elements are deleted from the state
- An action is carried out if it takes the process 'closer' to the goal.

2. What strategies for generating optimal plans do you know? (2)

Ans : Strategies for generating optimal plans

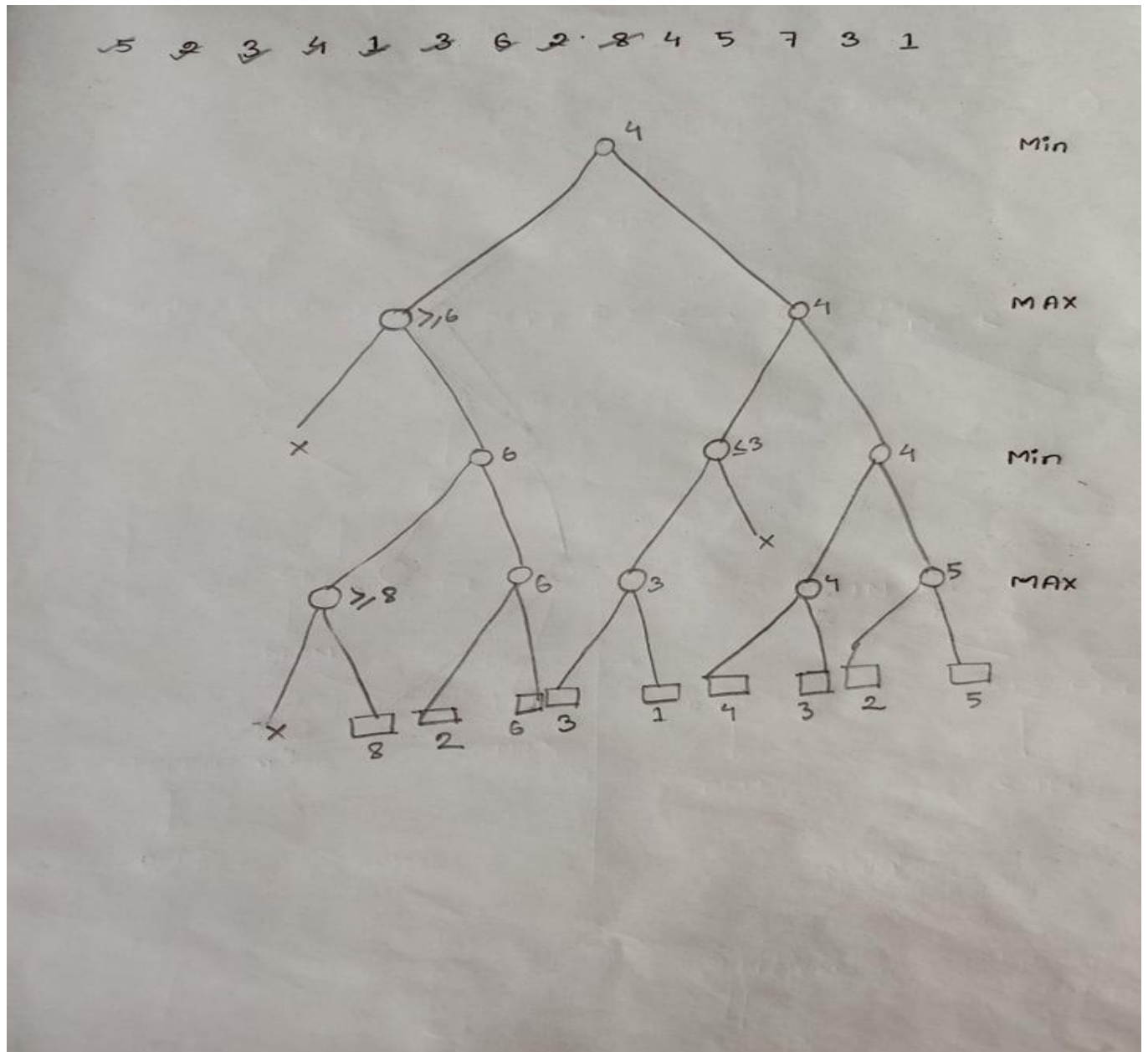
- Forward state space search [$IS \rightarrow GS$]
- Backward state space search [$IS \leftarrow GS$]
- Bidirectional search [$IS \rightarrow \dots \leftarrow GS$]

3. What is a cooperative game? (1)

Ans : In cooperative games, players don't compete against each other. Instead they have a common goal so players either win or lose together. Cooperative games require users to coordinate, cooperate and communicate with an AI agent in ways that their actions are deemed interpretable and interpret the actions of the AI agent.

4. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [5, 2, 3, 4, 1, 3, 6, 2, 8, 4, 5, 7, 3, 1] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MIN makes the first move, and nodes are generated from right to left. (4)

Ans :



111

1. Explain the advantages and disadvantages of MINIMAX search strategy. (3)

Advantages:

- **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if it exists), in the finite search tree.

- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is the branching factor of the game-tree, and m is the maximum depth of the tree.

Disadvantages: The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of game has a huge branching factor, and the player has lots of choices to decide. Also a disadvantage of the minimax algorithm is that each board state has to be visited twice: one time to find its children and a second time to evaluate the heuristic value.

2. What makes the search space complicated in game playing? (2)

Ans: Mathematical game theory, a branch of economics, views any multi agent environment as a game, provided that the impact of each agent on the others is “significant,” regardless of whether the agents are cooperative or competitive.¹ In AI, the most common games are of a rather specialized kind—what game theorists call deterministic, turn-taking, two-player, ZERO-SUM GAMES zero-sum games of perfect information (such as chess). In our terminology, this means PERFECT INFORMATION deterministic, fully observable environments in which two agents act alternately and in which the utility values at the end of the game are always equal and opposite. For example, if one player wins a game of chess, the other player necessarily loses.

3. What is a planner? (1)

A planner is usually a goal-based agent; ✓ A planner takes advantage of the structure and specific features of the problem domain to formulate the actions; ✓ A planner follows the rule: Activities should be preconditioned; ✓ A planner decomposes a problem into independent subproblems, and it achieves the goal by satisfying subgoals.

4. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates ‘Airport’, ‘Cargo’ and ‘Plane’, and consider generally accepted rules and notations for such a problem. (4)

A1	A2	A3
C1 C2	C3 C4	C5 C6
P1 P2	P3 P4	P5 P6

Initial State

A1	A3
C4 C5	C3 C2

Goal State

Action 1:

Load (C1, P1, A1)
Load (C5, P5, A3)
Load (C2, P1, A1)

Action 2:

Fly (P1, A1, A2)
Fly (P5, A3, A2)

Action-3

Unload (C2, P1, A2)
Unload (C5, P5, A2)
Load (C4, P4, A2)

Action 4

Fly (P4, A2, A3)
Fly (P1, A2, A3)

Action 5 Unload (C1, P1, A3)
 Unload (C4, P4, A3)

state

$$\begin{aligned}
 & \text{At}(A_1, C_1) \wedge \text{At}(A_1, C_2) \wedge \text{At}(A_1, P_1) \wedge \text{At}(A_2, P_2) \wedge \text{At}(A_2, C_3) \\
 & \wedge \text{At}(A_2, C_4) \wedge \text{At}(A_2, P_3) \wedge \text{At}(A_2, P_4) \wedge \text{At}(A_3, C_5) \\
 & \wedge \text{At}(A_3, C_6) \wedge \text{At}(A_3, P_5) \wedge \text{At}(A_3, P_6) \wedge \text{In}(P_1, C_1) \\
 & \wedge \text{In}(P_1, C_2) \wedge \text{In}(P_5, C_5) \wedge \text{At}(A_2, P_2) \wedge \text{At}(A_2, P_5) \\
 & \wedge \text{In}(A_2, C_2) \wedge \text{In}(A_2, C_5) \wedge \text{At}(A_3, P_4) \wedge \text{At}(A_3, P_2)
 \end{aligned}$$

1. Explain how MINIMAX values are assigned to nodes of game trees. (2)**Ans:**

In game theory, minimax is a decision rule used to minimize the worst-case potential loss; in other words, a player considers all of the best opponent responses to his strategies, and selects the strategy such that the opponent's best strategy gives a payoff as large as possible.

The Minimax Algorithm moves in depth-first fashion down the tree until it reaches a terminal node (*i.e.* someone wins the game) or a predetermined depth limit. Depth limits are set for games involving complex search spaces, in which it would not be feasible to search the entire network of possible moves within a reasonable amount of time. Once it reaches a terminal node or depth limit, the *Utility Calculation* function is called, and a resulting utility value for that particular terminal board is calculated.

2. How time constraint affects game playing? (2)**Ans:**

Time is an important factor in decision making, one we can't forget about when designing any kind of system. With gamification as in games, we have the ability to use it to our advantage by imposing time constraints on systems that would otherwise not have them.

By giving shorter time limits to achieve tasks, we limit the number of options that can be considered by people, but by doing so may promote the most practical options to rise to the top. Forcing people into the here and now will make those decisions feel more concrete and real, giving them more immediate meaning.

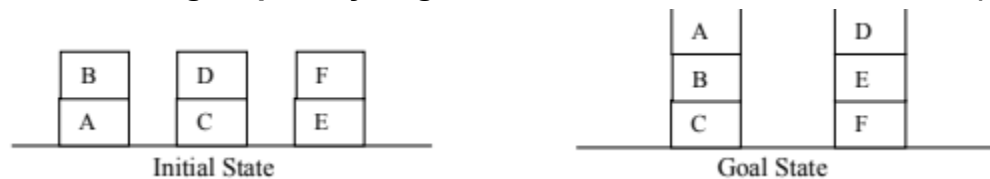
Giving longer time periods allows people to craft more thoughtful ideas and come to more creative decisions, but can also lead to people dismissing good ideas based on personal bias or other external factors.

3. Why is a planner called a goal based agent? (2)**Ans:**

A goal-based agent has an agenda, you might say. It operates based on a goal in front of it and makes decisions based on how best to reach that goal. Unlike a simple reflex agent that makes decisions based solely on the current environment, a goal-based agent is capable of thinking beyond the present moment to decide the best actions to take in order to achieve its goal. In this regard, a goal-based agent operates as a search and planning function, meaning it targets the goal ahead and finds the right action in

order to reach it. This helps a goal-based agent to be proactive rather than simply reactive in its decision-making. That is why a planner is called a goal based agent.

4. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in number of actions over time. (4)



Ans:

115

1. Explain the concept of action schemas using an example from Tower Building in Blocks World problem. (2)

Ans:

c) Action Schemas:

i) Action: Move(b1, x, b2)

Precondition: $\text{Block}(b1) \wedge \text{Block}(b2) \wedge \text{On}(x, b1) \wedge \text{Clear}(b1) \wedge \text{Clear}(b2)$

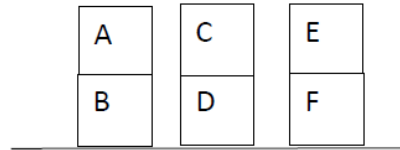
Effect: $\text{On}(b2, b1) \wedge \neg \text{On}(x, b1) \wedge \text{Clear}(x) \wedge \neg \text{Clear}(b2)$

ii) Action: MoveToTable(b1, b2)

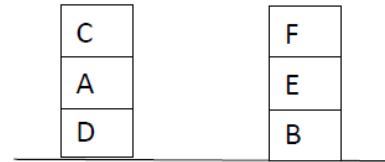
Precondition: $\text{Block}(b1) \wedge \text{Block}(b2) \wedge \text{On}(b2, b1) \wedge \text{Clear}(b1)$

Effect: $\text{On}(\text{Table}, b1) \wedge \neg \text{On}(b2, b1) \wedge \text{Clear}(b2)$

Example:



Initial State



Goal State

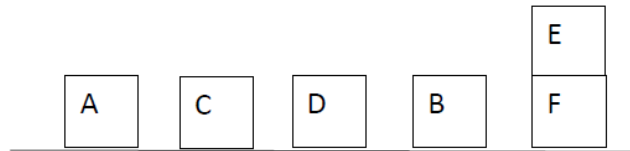
Action 1:

MoveToTable(A, B)

MoveToTable(C, D)

State 1: [Predicate 'Block' has been omitted.]

$\text{On}(\text{Table}, B) \wedge \text{On}(\text{Table}, D) \wedge \text{On}(\text{Table}, F) \wedge \text{On}(B, A) \wedge \text{On}(D, C) \wedge \text{On}(F, E) \wedge$
 $\text{Clear}(A) \wedge \text{Clear}(C) \wedge \text{Clear}(E) \wedge \text{On}(\text{Table}, A) \wedge \text{On}(\text{Table}, C) \wedge \text{Clear}(D) \wedge$
 $\text{Clear}(B)$



Goal test: False

Goal State

State 2: [Predicate 'Block' has been omitted.]

$\text{On}(\text{Table}, B) \wedge \text{On}(\text{Table}, D) \wedge \text{On}(\text{Table}, F) \wedge \text{On}(B, A) \wedge \text{On}(D, C) \wedge \text{On}(F, E) \wedge$
 $\text{Clear}(A) \wedge \text{Clear}(C) \wedge \text{Clear}(E) \wedge \text{On}(\text{Table}, A) \wedge \text{On}(\text{Table}, C) \wedge \text{Clear}(D) \wedge \text{Clear}(B)$

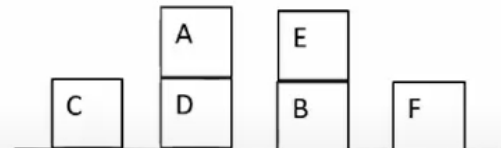
$\text{On}(D, A) \wedge \text{On}(B, E) \wedge \text{Clear}(F)$

Goal test: False

Action 2:

Move(A, Table, D)

Move(E, F, B)



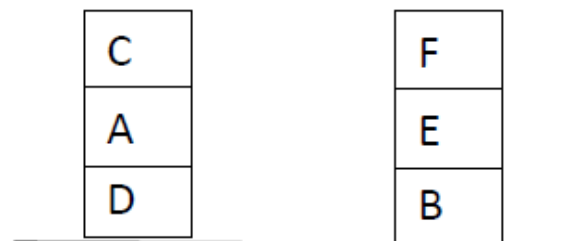
State 2

State 3

Action 3:

Move(C, Table, A)

Move(F, Table, E)



Goal Test : True

2. How a planner exerts intelligence through domain feature consideration? (2)

Ans:

5) Planner: An intelligent agent

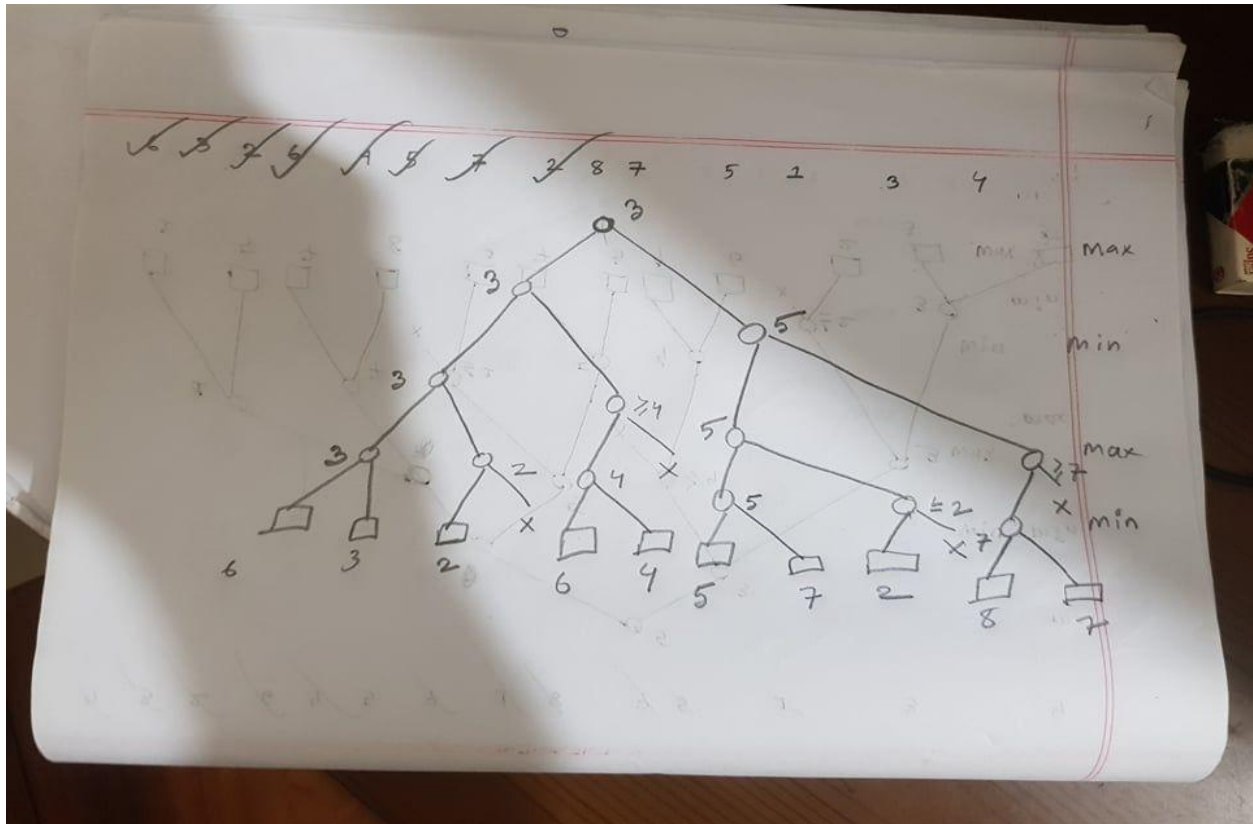
- ✓ A planner is usually a goal-based agent;
- ✓ A planner takes advantage of the structure and specific features of the problem domain to formulate the actions;
- ✓ A planner follows the rule: Activities should be preconditioned;
- ✓ A planner decomposes a problem into independent subproblems, and it achieves the goal by satisfying subgoals.

3. What is quiescence search? (2)

Ans: Quiescence search is an algorithm typically used to extend search at unstable nodes in minimax game trees in game-playing computer programs. It is an extension of the evaluation function to defer evaluation until the position is stable enough to be evaluated statically, that is, without considering the history of the position or future moves from the position. It mitigates the effect of the horizon problem faced by AI engines for various games like chess and Go.

A quiescence search attempts to emulate this behavior by instructing a computer to search "volatile" positions to a greater depth than "quiet" ones to make sure there are no hidden traps and to get a better estimate of its value.

4. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [6, 3, 2, 6, 4, 5, 7, 2, 8, 7, 5, 1, 3, 4] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MAX makes the first move, and nodes are generated from left to right. (4)



Root e 5 hobe

116

1. Explain two important aspects of MINIMAX search algorithm. (2)

Ans-

- 1) Returns an optimal strategy; Opener always wins if the players have equal opportunity and implement the strategy without mistake.
- 2) The strategy is a depth-first one; Time cost is impractical for real games.
- 3) Improvements can be proposed using heuristics and pruning.
- 4) The algorithm can be extended for more than 2-player games with a vector of utility values for each node reflecting each player's view.
- 5) In a multiplayer game, alliances / coalitions are made and broken as the game proceeds for cooperation among some against other(s)

2. Why is game playing considered a complicated search problem? (2)

Ans:

Game Playing is an important domain of artificial intelligence. Games don't require much knowledge; the only knowledge we need to provide is the rules, legal moves and the conditions of winning or losing the game.

Both players try to win the game. So, both of them try to make the best move possible at each turn. Searching techniques like BFS(Breadth First Search) are not accurate for this as the branching factor is very high, so searching will take a lot of time. So, we need another search procedures that improve –

- Generate procedure so that only good moves are generated.
- Test procedure so that the best move can be explored first.

The most common search technique in game playing is the Minimax search procedure. It is depth-first depth-limited search procedure. It is used for games like chess and tic-tac-toe.

3. How 'precondition' of actions are represented with PDDL? (2)

Ans:

An action schema is used to generate instances of the action.

An action schema contains:

- An action name with a parameter list.
- A 'precondition' as a conjunction of function-free and positive literals, where the arguments of the predicates are taken from the parameter list;
- An 'effect' as a conjunction of function-free literals, where also the arguments of the predicates are taken from the parameter list .

The positive and negative literals of an 'effect' are stored in the 'Add list' and 'Delete list' respectively

4. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates 'Airport', 'Cargo' and 'Plane', and consider generally accepted rules and notations for such a problem. (4)



1. Explain the common factors considered for improving the performance of the MINIMAX algorithm. (2)

Ans-

- 1) Database of experience like 'best opening move', 'great winning [/losing] combination', ...
- 2) Maintaining transposition table that marks repeated states
- 3) Forward pruning, that excludes equivalent states
- 4) Singular expansion, where only 'clearly better' moves are considered

2. How game playing may be both cooperative and competitive? (2)

Ans-

Cooperative games offer a new way to play. competitive games are zero-sum games: One player can win only if another loses. In cooperative games, players don't compete against each other. Instead they have a common goal so players either win or lose together.

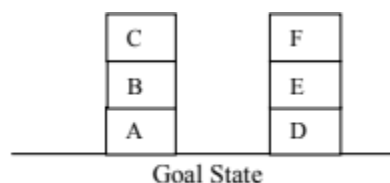
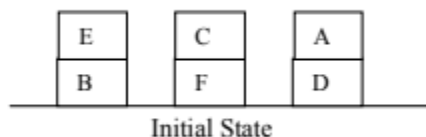
Games may be competitive as well as cooperative; Mathematical game theory considers the impact of both types of agents, cooperative and competitive, important.

3. What is the purpose of action schemas? (2)

Ans-

- i) Action: Load(c, p, a) Precondition: Cargo(c) Plane(p) Airport(a) At(a, c) At(a, p)
Effect: At (a, c) In(p, c)
- ii) Action: Unload(c, p, a) Precondition: Cargo(c) Plane(p) Airport(a) At(a, p) In(p, c)
Effect: At (a, c) In(p, c)
- iii) Action: Fly(p, from, to) Precondition: Plane(p) Airport(from) Airport(to) At(from, p)
Effect: At (from, p) At(to, p)

4. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in time over the number of actions. (4)



1. Explain how 'effect' of an action is represented and used in planning. (2)

ANS; An 'effect' of an action is represented as a conjunction of function-free literals, where also the arguments of the predicates are taken from the parameter list .

The positive and negative literals of an 'effect' are stored in the 'Add list' and 'Delete list' respectively.

2. What makes a planner intelligent? (2)

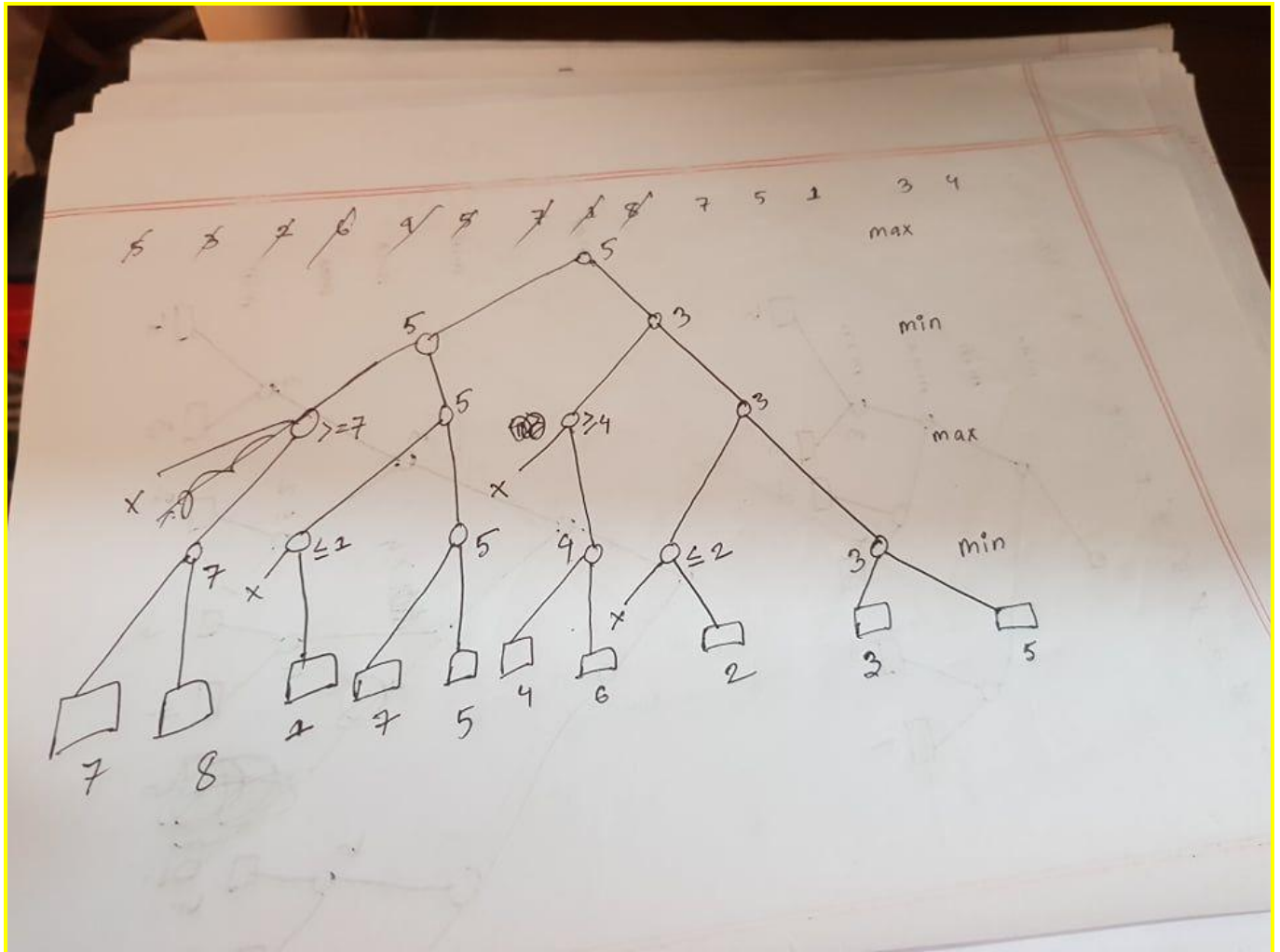
ANS: a planner is intelligent because:

- ✓ A planner is usually a goal-based agent;
- ✓ It takes advantage of the structure and specific features of the problem domain to formulate the actions;
- ✓ A planner follows the rule: Activities should be preconditioned;
- ✓ It decomposes a problem into independent subproblems, and it achieves the goal by satisfying subgoals.

3. Why are players of a two-player board game called MAX and MIN? (2)

ANS: In two-player board game one player is called MAX because it searches for the sequence of moves (Plan) that leads to a terminal with maximum possible utility value, even if MIN plays in the best way. And the other one is called MIN because it searches the terminal with the minimum possible utility, which is opposite of MAX.

4. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [5, 3, 2, 6, 4, 5, 7, 1, 8, 7, 5, 1, 3, 4] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MAX makes the first move, and nodes are generated from right to left. (4)



120

1. Explain the advantages of MINIMAX search strategy. (2)

Ans -

The popularity of Minimax is based on the fact that it takes into account all the possible moves that players can take at any given time during the game. This enables the algorithm to minimize the opponent's advantage while simultaneously maximize the agent's advantage at every turn the AI gets to play.

2. What do you know about multi-agent environments in game playing? (2)

Ans

Multiple agent environment: More than one agent; Usually, their goals are in conflict; Agents plan against each other; Adversarial search.

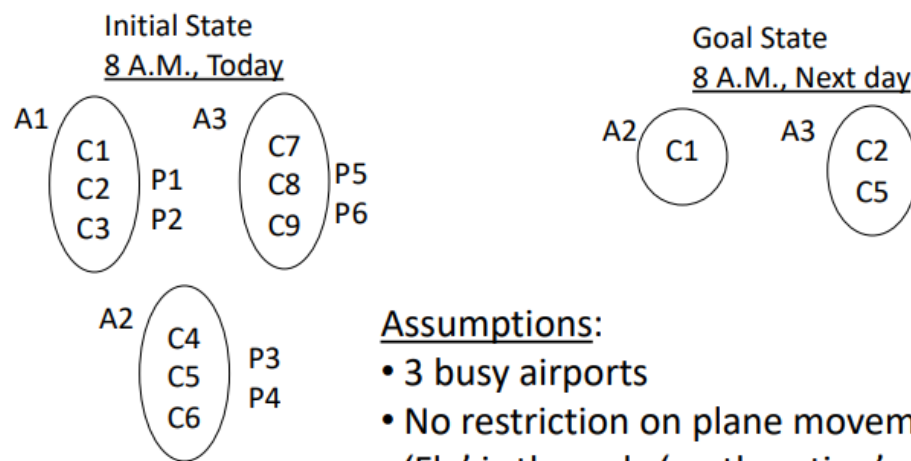
Multi-agent systems (MAS) are a core area of research of contemporary artificial intelligence. A multi-agent system consists of multiple decision-making agents which interact in a shared environment to achieve common or conflicting goals.

3. How goal states are represented in planning problems? (2)

Ans-

A) Representation of the problem

a) Schematic representation



Assumptions:

- 3 busy airports
- No restriction on plane movement
- 'Fly' is the only 'costly action'
- Any plane can reach another port directly or via the 3rd by the given time

02-Aug-21

1

4. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates 'Airport', 'Cargo' and 'Plane', and consider generally accepted rules and notations for such a problem. (4)



1. What are the disadvantages of MINIMAX strategy and how can those be reduced? (4)

Ans : The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. Time cost is impractical for real games that have a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from using heuristics and pruning.

The algorithm can be extended for more than 2-player games with a vector of utility values for each node reflecting each player's view. In a multiplayer game, alliances / coalitions are made and broken as the game proceeds for cooperation among some against other(s).

1. Using 'cutoff test' and heuristic evaluation function

- a. To have a reduced search space 'cutoff tests' are applied in place of 'terminal tests':
 - Simple: Set a fixed depth; Not so good.
 - Sophisticated: quiescence search; [A quiescent state does not exhibit wild swing in near future.]
- b. To apply on states at cutoff depths, utility functions are replaced with heuristic functions, which return expected utility of states. [In Chess, material values like pawn = 1, knight/bishop = 3, queen = 9, etc. are used.]

2. Using alpha-beta pruning:

Cutting off in compliance with already calculated minimax values, that is, values of

- i. best choice for the maximizing player,
- ii. best choice for the minimizing player

2. How 'precondition' makes a planner intelligent? (2)

3. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in number of actions over time. (4)



1. What is an action schema and how is it used to carry out an action? (4)

Ans:

- An action schema is used to generate instances of the action.
- An action schema contains:
 - An action name with a parameter list;
 - A 'precondition' as a conjunction of function-free and positive literals, where the arguments of the predicates are taken from the parameter list;
 - An 'effect' as a conjunction of function-free literals, where also the arguments of the predicates are taken from the parameter list .
- The positive and negative literals of an 'effect' are stored in the 'Add list' and 'Delete list' respectively.

How it is used to carry out an action:

- ✓ An action can be carried out on a state if the precondition holds on that state with appropriate substitutions;
- ✓ If an action is carried out, the Add-list elements are included in the state, while the Delete- list elements are deleted from the state;
- ✓ An action is carried out if it takes the process 'closer' to the goal.

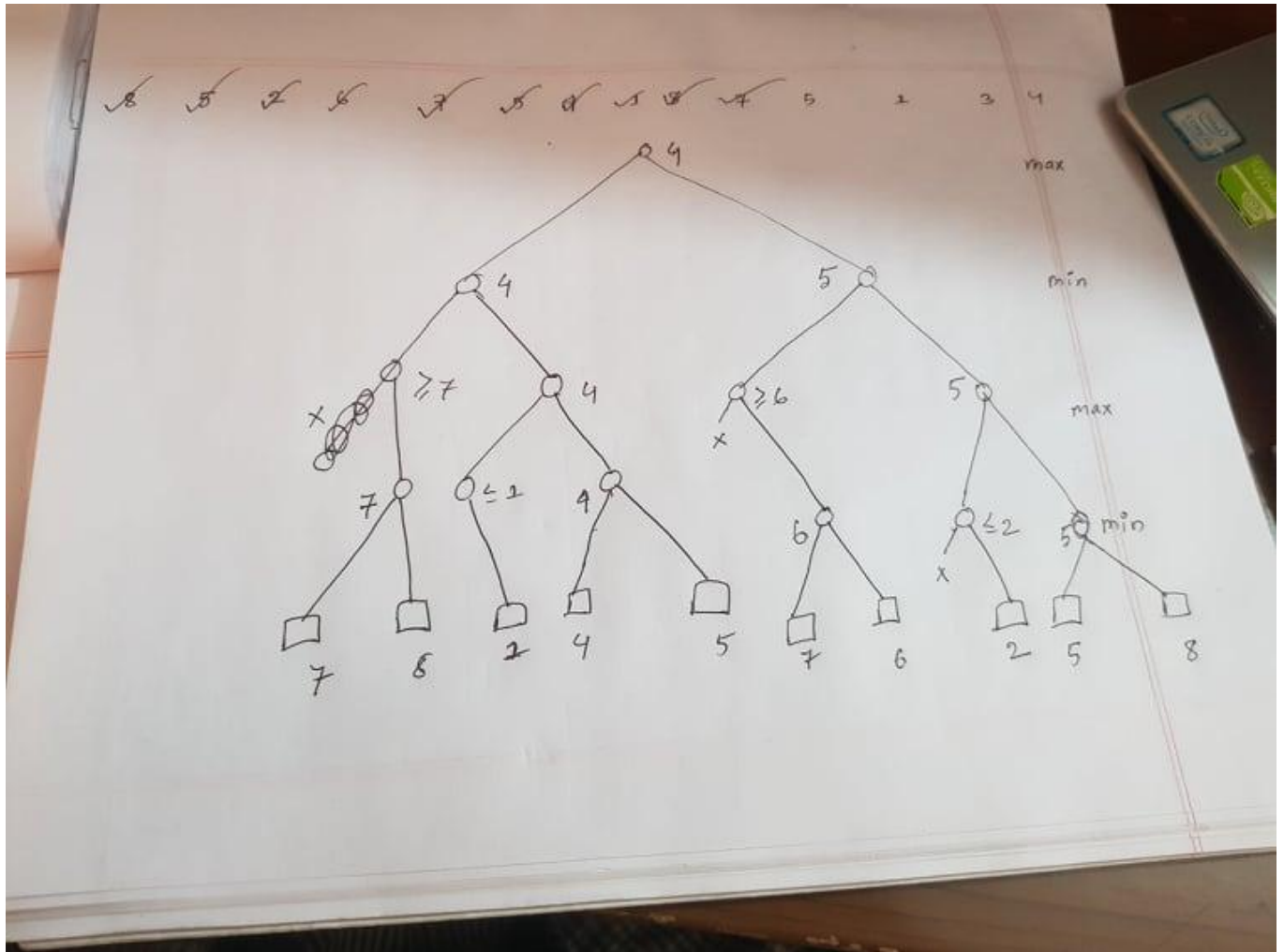
2. Elaborate the concept of quiescence search. (2)

Ans:

A quiescence search attempts to emulate a behavior to have enough intuition to decide whether to abandon a bad-looking move, or search a promising move to a great depth by instructing a computer to search "volatile" positions to a greater depth than "quiet" ones to make sure there are no hidden traps and to get a better estimate of its value.

For example: Most chess programs, at the end of the main search, perform a more limited quiescence search, containing fewer moves. The purpose of this search is to only evaluate "quiet" positions, or positions where there are no winning tactical moves to be made.

3. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [8, 5, 2, 6, 7, 5, 4, 1, 8, 7, 5, 1, 3, 4] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MAX makes the first move, and nodes are generated from right to left. (4)



Root e 5 hobe

125

1. How game playing is converted to search problems and what peculiarities does such a search possess? (4)

Ans :

2. Distinguish between total-order and partial-order plans. (2)

Ans :

Total order plan: No action can be executed in parallel to each other; By contrast, total-order planning maintains a total ordering between all actions at every stage of planning.

Partial order plan: Some actions may be executed in parallel; Partial-order planning is an approach to automated planning that maintains a partial ordering between actions and only commits ordering between actions when forced to, that is, ordering of actions

is partial. Also this planning doesn't specify which action will come out first when two actions are processed. By contrast, total-order planning maintains a total ordering between all actions at every stage of planning.

3. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates 'Airport', 'Cargo' and 'Plane', and consider generally accepted rules and notations for such a problem. (4)



148

1. What is alpha-beta pruning and how a pruned game tree is constructed? (4)

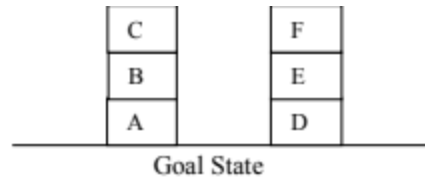
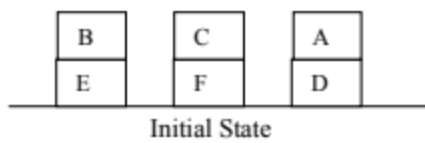
Ans : The problem with minimax search is that the number of game states it has to examine is exponential in the depth of the tree. Unfortunately, we can't eliminate the exponent, but it turns out we can effectively cut it in half. The trick is that it is possible to compute the correct minimax decision without looking at every node in the game tree. That is, we can borrow the idea of pruning to eliminate large parts of the tree from consideration. The particular technique we examine is called alpha-beta pruning. When applied to a standard minimax tree, it returns the same move as minimax would, but prunes away branches that cannot possibly influence the final decision.

2. Elaborate the concept of imperfect information considered in game playing. (2)

Ans : A game of incomplete information is a game where the players do not have common knowledge of the game being played. This idea is tremendously important in capturing many economic situations, where a variety of features of the environment may not be commonly known.

3. Generate an optimal plan through Forward State Space search using PDDL for the planning problem in the Blocks world presented below. Omit the predicate 'Block', and give priority to gain in time over the number of actions. (4)

Ans :



151

1. How states and actions are represented using PDDL? (4)

Ans:

Representation of states using PDDL:

- Conjunction of function-free, positive and ground literals of FOL
- Closed world assumption: When fully specified, not mentioned is equivalent to False
- The goal state is usually partially specified
- If a current state contains the goal state, then the goal is considered achieved.

Representation of actions using PDDL:

- An action schema is used to generate instances of the action.
- An action schema contains:
 - An action name with a parameter list
 - A 'precondition' as a conjunction of function-free and positive literals, where the arguments of the predicates are taken from the parameter list
 - An 'effect' as a conjunction of function-free literals, where also the arguments of the predicates are taken from the parameter list .
- The positive and negative literals of an 'effect' are stored in the 'Add list' and 'Delete list' respectively.

2. Write a short note on 'Decomposition in designing a plan'. (2)

Ans :

3. Construct a pruned game tree using Alpha-Beta pruning. Take the sequence, [1, 3, 4, 5, 7, 4, 6, 2, 1, 3, 5, 4, 8, 4, 6] of MINIMAX values for the nodes at the cutoff depth of 4 plies. Assume that branching factor is 2, MIN makes the first move, and nodes are generated from left to right. (4)

Ans :

[Values returned by Utility function / Payoff function/
Objective function; MAX's point of view has been reflected.]

iv) Strategies of Players

- ✓ MAX searches for the sequence of moves (Plan) that leads to a terminal with maximum possible utility value, even if MIN plays in the best way;
- ✓ MIN searches for the opposite, that is, terminal with minimum possible utility.

2. How do you explain the closed world assumption? (2)

Ans :

The Closed World Assumption (CWA) is the assumption that that what is not known to be true, is false, so that absence of information is interpreted as negative information. The closed-world assumption (CWA), in a formal system of logic used for knowledge representation, is the presumption that a statement that is true is also known to be true. Therefore, conversely, what is not currently known to be true, is false.

3. Generate an optimal plan using PDDL and Forward State Space search for the Air Cargo Transportation problem given below. Omit predicates 'Airport', 'Cargo' and 'Plane', and consider generally accepted rules and notations for such a problem. (4)

A1	A2	A3
C1 C2	C3 C4	C5 C6
P1P2	P3 P4	P5 P6

Initial State

A2	A3
C1 C5	C4 C2

Goal State

Ans :

Nicher page e answer + Jegula kete kete disi, oigulao likhe likhe kete dibi

Init state:

$$\begin{aligned} & \neg At(A1, C1) \wedge \neg At(A1, C2) \wedge \neg At(A1, P1) \wedge \neg At(A1, P2) \wedge \\ & At(A2, C3) \wedge \neg At(A2, C4) \wedge At(A2, P3) \wedge \neg At(A2, P4) \wedge \\ & \neg At(A3, C5) \wedge At(A3, C6) \wedge \neg At(A3, P5) \wedge At(A3, P6) \\ & \text{Action Group 1: } \frac{\wedge In(P1, C1) \wedge In(P2, C2) \wedge In(P4, C4) \wedge In(P5, C5)}{1} \\ & \frac{At(A2, P1) \wedge \neg At(A3, P2) \wedge At(A3, P4) \wedge At(A2, P5) \wedge At(A2, C1) \wedge \neg At(A2, C5)}{2} \\ & \wedge At(A3, C4) \wedge At(A3, C2) \end{aligned}$$

Action Group 1: Load(C1, P1, A1) $\wedge$ Load(C4, P4, A2)
 $\wedge$ Load(C2, P2, A1) $\wedge$ Load(C5, P5, A3)
Goal test = false

Action Group 2: Fly(P1, A1, A2) $\wedge$ Fly(P2, A1, A3) $\wedge$
Fly(P4, A2, A3) $\wedge$ Fly(P5, A3, A2)

Goal test = false

Action Group 3: Unload(C1, P1, A2) $\wedge$ Unload(C5, P5, A2)
 $\wedge$ Unload(C4, P4, A3) $\wedge$ Unload(C2, P2, A3)

Goal test = true